Balanced Expected-Pyramid KVPress Compression for Training-Free Pythia-70M Inference

Daorui Jiang **Hao Ding** **Censhuo Zhang**
Shanghai Jiao Tong University
https://github.com/SJTU-huaxing/llm-hw-team.git

Abstract

KV cache compression reduces autoregressive inference memory, but aggressive pruning often trades quality for speed. We study this tradeoff on EleutherAI/Pythia-70M with NVIDIA KVPress, no training, and fp32 eager attention. Beyond Dense, StreamingLLM, SnapKV, PyramidKV, ExpectedAttention, ObservedAttention, and TOVA baselines, we iteratively test hybrid and layer-budget variants. The selected aggressive-compression method is Expected Soft Pyramid: it keeps ExpectedAttention as the token scorer and applies a mild low-to-high layer budget schedule. At a 132-token target cache for a 512-token context, it lies on the Pareto frontier and has the best equal-rank balance over average perplexity, TPOT, and decode-attention FLOPs. We also report that adding observed-attention residuals often hurts WikiText quality, which makes the conservative ExpectedAttention-based variant the most reliable innovation.

1 Introduction

The KV cache stores one key and value vector per layer and token, so long-context decoding stresses GPU memory and attention bandwidth. The assignment asks us to combine ideas from four levels: attention mechanisms, per-layer KV eviction, inter-layer cache allocation, and macro inference architecture. We keep the model fixed, use no training, and use KVPress [NVIDIA, 2025] to hook Transformers during prefill.

Pythia-70M [Biderman et al., 2023] uses GPT-NeoX attention modules and partial RoPE, while several KVPress utilities assume LLaMA-like module names and full-dimensional RoPE. Our adapter only supplies these structural aliases and the partial-RoPE correction needed for ExpectedAttention. All compression decisions are implemented as KVPress-style presses that gather existing KV tensors along the sequence dimension.

Our goal is therefore not to design a new model architecture, but to identify an inference-only policy that is useful under the assignment constraints. We treat quality and latency as a joint objective: a method that is slightly faster but severely increases perplexity is not acceptable, and a method that preserves quality while adding expensive bookkeeping is also unattractive at the small Pythia-70M scale. This motivates a conservative innovation: keep the strongest token-importance signal from KVPress, then only alter how the cache budget is distributed across layers.

2 Method

Table 1 summarizes how the implementation covers the assignment layers. Official baselines include attention sink retention [Xiao et al., 2023], observed attention from SnapKV/PyramidKV [Li et al., 2024, Cai et al., 2024], H2O-style heavy-hitter scoring [Zhang et al., 2023], and TOVA [Oren et al., 2024].

Preprint.

Table 1: Mapping from acceleration layers to implemented components.

Layer	Components
Attention mechanism	ExpectedAttention, ObservedAttention/TOVA, value norm
Per-layer compression	StreamingLLM, SnapKV, PyramidKV, observed/top-k scorers
Inter-layer compression	Pyramid budgets, lazy gate, expected soft budget ablation
Macro architecture	KVPress pipeline, prefill-once compression, unified timing

Expected Soft Pyramid uses the ExpectedAttention score from KVPress but changes the budget per layer:

$$B_\ell = \mathrm{round}(B\alpha_\ell), \quad \alpha_\ell \in [1.25, 0.75],$$

where α_ℓ decreases linearly from lower to higher layers and averages to one. This keeps the global target cache size unchanged while testing the common observation that lower layers can benefit from broader lexical context. Sink tokens are always protected. We also test Hybrid Expected-Observed,

$$s = 0.55\,\hat{s}_{\mathrm{expected}} + 0.30\,\hat{s}_{\mathrm{observed}} + 0.15\,\hat{s}_{\mathrm{vnorm}},$$

where each score is min-max normalized per batch and head. Sink tokens and a 32-token recent window receive a protection boost before top-k selection. Residual and soft-pyramid hybrid variants are retained as ablations.

The implementation follows the same prefill-once abstraction for every method. Given a context of length L, KVPress first builds the dense cache, the press scores the cached positions, and the resulting indices are gathered from the key and value tensors. During decoding, no further model weights or attention equations are changed; the model simply attends to the shortened cache. For Pyramid-style methods, different layers may have different retained lengths. The Pythia adapter records these per-layer lengths and slices masks accordingly, so the reported memory and FLOPs use the actual cache shape rather than the nominal target budget.

We considered more aggressive innovations, including expected-observed residual scores and a residual soft pyramid. These variants are meaningful because observed attention can identify heavy hitters in the concrete prompt, while value-norm scores can favor tokens with larger downstream contribution. However, early experiments showed that forcing recent or observed-heavy tokens can overfit to local continuation patterns. Expected Soft Pyramid was chosen as the main custom method because it changes only the inter-layer allocation and leaves the more robust ExpectedAttention ordering intact.

3 Experiments

We evaluate 'EleutherAI/pythia-70m' on WikiText-2 test [Merity et al., 2017] and one PG-19 test stream sample [Rae et al., 2020]. For PPL, a 512-token context is prefetched and compressed before scoring the next 128 continuation tokens. For generation, a 512-token PG-19 prompt generates 32 tokens. The decode attention FLOPs proxy is $\sum_{\ell,t} 4HDL_{\ell,t}$. Reported runs use an NVIDIA GeForce RTX 5060 Laptop GPU.

We first screen new baselines and hybrids on 256-token contexts and target cache sizes 96, 128, and 192. The strongest new official baselines are ObservedAttention and TOVA, so the final table keeps them alongside the original methods. The balanced selection rule filters methods with kept fraction above 0.5 and ranks the remaining methods by average PPL, TPOT, and FLOPs ratio with equal weight.

All scripts write CSV and JSON files with the KVPress version, Transformers version, target cache size, compression ratio, per-layer cache lengths, and CUDA memory. This is important because two methods can share the same average kept fraction while having different layer distributions. For example, ExpectedAttention at target 132 keeps 132 tokens in every layer, whereas Expected Soft Pyramid keeps $[165, 152, 139, 125, 112, 99]$ tokens across the six Pythia layers. Both average to 132, but the latter spends more budget in early layers and less in later layers.

Table 2: Target cache 132. PPL is averaged over WikiText-2 and PG-19; generation is on PG-19.

Policy	Avg PPL	Kept	TPOT	FLOPs
Dense	21.344	1.000	0.006071	1.000
ExpectedAttention	26.326	0.258	0.006373	0.281
Expected Soft Pyramid	28.342	0.258	0.006066	0.281
Expected+Pyramid	35.066	0.259	0.006056	0.282
Hybrid Expected-Observed	37.870	0.258	0.006080	0.281
Hybrid Soft Pyramid	38.884	0.258	0.006064	0.281
SnapKV	42.340	0.258	0.005959	0.281
StreamingLLM	46.148	0.258	0.006148	0.281
TOVA	53.386	0.258	0.006146	0.281

Table 3: Balanced selections across target cache sizes.

Target	Selected policy	Avg PPL	TPOT	FLOPs
132	Expected Soft Pyramid	28.342	0.006066	0.281
192	ExpectedAttention	22.407	0.006006	0.395
256	Hybrid Expected-Observed	26.329	0.006105	0.516

4 Analysis and Limitations

The main result is not that a more complicated scorer always wins. ExpectedAttention is clearly the quality anchor, especially on WikiText-2. The successful innovation is conservative: Expected Soft Pyramid changes the inter-layer budget while keeping the ExpectedAttention scorer intact. It gives up about 2 average PPL relative to ExpectedAttention at target 132, but improves TPOT and remains on the Pareto frontier. In contrast, adding observed-attention residuals improves some PG-19 cases but tends to damage WikiText quality, so we report those variants as negative ablations rather than the recommended method.

The difference between WikiText-2 and PG-19 is instructive. WikiText-2 passages are shorter and more locally coherent, so removing globally useful context hurts more visibly; ExpectedAttention is best because it estimates future positions instead of only looking at recent observed attention. PG-19 contains long book-like text, where local repetition and narrative continuity make sink, recent, and observed-heavy policies less harmful. This explains why several observed or hybrid methods have competitive PG-19 PPL but worse average PPL. The final selection rule deliberately averages both datasets so that the method does not specialize to one text style.

The speed results should be interpreted as a proxy for cache efficiency rather than a universal deployment claim. Pythia-70M is small enough that Python overhead, CUDA synchronization, and pipeline setup can be comparable to the saved attention computation. For this reason, we report TTFT, TPOT, throughput, peak memory, kept fraction, and decode-attention FLOPs together. The consistent FLOPs reduction from roughly 1.0 to 0.281 at target 132 confirms that the compressed methods reduce the theoretical attention work, while the TPOT differences show the practical overhead of each scoring and layer-budget choice.

The evaluation is intentionally small and reproducible: one model, no training, two datasets, fixed seeds, and one GPU. Timing differences are small at Pythia-70M scale, so the paper reports exact CSVs rather than over-claiming hardware speedups. We do not implement MQA/GQA/MLA or YOCO-style cross-layer sharing because those require architecture or weight changes outside the no-training setting. KVPress SimLayerKV is excluded because its full-RoPE assumption is incompatible with GPT-NeoX partial RoPE without further changes.

Reproducibility is handled at the project level. The Conda environment pins KVPress 0.5.3, Transformers 4.57.6, and CUDA PyTorch; the README lists the exact commands for smoke tests, PPL evaluation, generation benchmarking, plot generation, and paper compilation. The repository also keeps the exploratory candidate files, so the negative residual experiments can be audited rather than hidden. This makes the final recommendation modest but traceable: use ExpectedAttention for best quality, use SnapKV when only TPOT matters, and use Expected Soft Pyramid when a balanced PPL-speed tradeoff under aggressive compression is preferred.

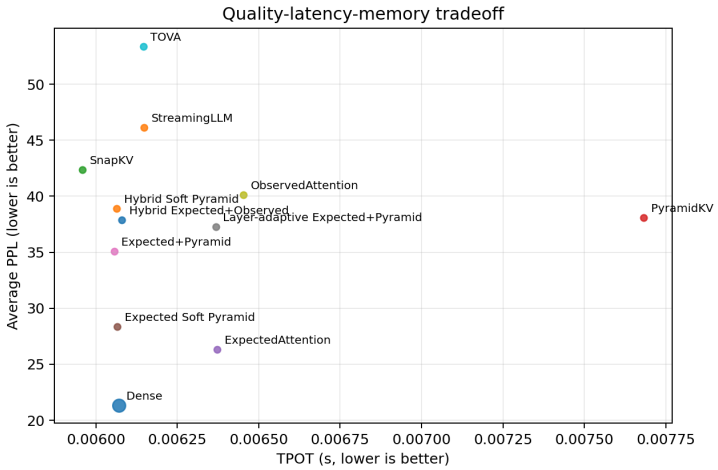

Figure 1: Average PPL versus TPOT at target cache 132.

5 Team Contributions

Daorui Jiang: method design, KVPress implementation, and final integration (40%). Hao Ding: experiments, benchmark runs, and result analysis (35%). Censhuo Zhang: paper writing, reproducibility documentation, and final checking (25%).

References

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. *International Conference on Machine Learning*, 2023.

Zefan Cai, Yichi Zhang, Bofei Gao, Yuliang Liu, Tianyu Liu, Keming Lu, Wayne Xiong, Yue Dong, Baobao Chang, Junjie Hu, and Wen Xiao. PyramidKV: Dynamic kv cache compression based on pyramidal information funneling. *arXiv preprint arXiv:2406.02069*, 2024.

Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Andrea Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: Llm knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.

NVIDIA. KVPress: Key-value cache compression for efficient long-context generation. `https://github.com/NVIDIA/kvpress`, 2025. Version 0.5.3.

Matanel Oren, Michael Hassid, Nir Yarden, Yossi Adi, and Roy Schwartz. Transformers are multi-state rnns. *arXiv preprint arXiv:2401.06104*, 2024.

Jack W. Rae, Anna Potapenko, Siddhant M. Jayakumar, Chloe Hillier, and Timothy P. Lillicrap. Compressive transformers for long-range sequence modelling. *International Conference on Learning Representations*, 2020.

Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. *arXiv preprint arXiv:2309.17453*, 2023.

Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. *arXiv preprint arXiv:2306.14048*, 2023.